

hw_3

January 4, 2026

1 Importiamo le librerie

```
[2]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
```

1.1 Definiamo le funzioni

```
[3]: def f(x, alpha):
    d = alpha.shape[0] - 1
    y = np.zeros_like(x)
    for i in range(d+1):
        y = y + alpha[i] * x ** i
    return y

def vandermonde(x, d):
    n = x.shape[0]
    X = np.zeros((n, d+1))
    for i in range(d+1):
        X[:, i] = x ** i
    return X

def SVD (X, y, tol=1e-3):
    n_features, p = X.shape
    U, s, VT = np.linalg.svd(X, full_matrices=False)
    alpha = np.zeros((p, ))

    for i in range(p):
        if s[i] > tol:
            uTy = U[:, i].T @ y
            v_i = VT[i, :]
            alpha = alpha + uTy / s[i] * v_i
    return alpha
```

```
def eqn(X, y):
    XtX = X.T @ X
    Xty = X.T @ y
    alpha_eqn= np.linalg.solve(XtX, Xty)
    return alpha_eqn
```

1.2 Regressione Semplice Random

```
[13]: def generate(n, d_true, d_model, ax, title):
    alpha_true = np.ones(d_true + 1)

    x = np.linspace(0, 1, n)

    sigma = 0.1
    e = np.random.normal(loc=0, scale=sigma, size=(n,))

    y_true = f(x, alpha_true)
    y = y_true + e

    X = vandermonde(x, d_model)

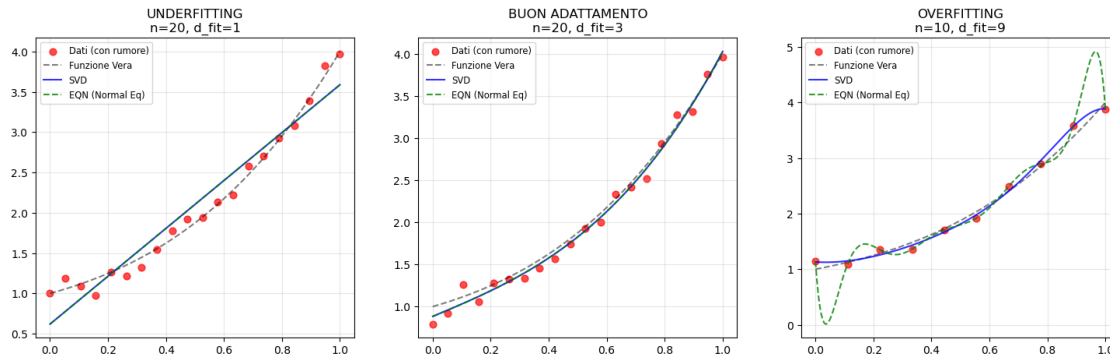
    alpha_svd = SVD(X, y)
    alpha_eqn = eqn(X, y)

    x_plot = np.linspace(0, 1, 200)
    y_true = f(x_plot, alpha_true)
    y_svd = f(x_plot, alpha_svd)
    y_eqn = f(x_plot, alpha_eqn)

    ax.scatter(x, y, color='red', label='Dati (con rumore)', s=40, alpha=0.7)
    ax.plot(x_plot, y_true, 'k--', label='Funzione Vera', alpha=0.5)
    ax.plot(x_plot, y_svd, 'b-', alpha=0.8, label='SVD')
    ax.plot(x_plot, y_eqn, 'g--', alpha=0.8, label='EQN (Normal Eq)')
    ax.set_title(f"{title}\nn={n}, d_fit={d_model}")
    ax.legend(fontsize='small')
    ax.grid(True, alpha=0.3)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

generate(n=20, d_true=3, d_model=1, ax=axes[0], title="UNDERFITTING")
generate(n=20, d_true=3, d_model=3, ax=axes[1], title="BUON ADATTAMENTO")
generate(n=10, d_true=3, d_model=9, ax=axes[2], title="OVERFITTING")
```



1.3 Database di Kaggle per Regressione Lineare Semplice

```
[14]: nome_file = 'Salary_dataset.csv'
df_student = pd.read_csv(nome_file)

print("\nPrime 5 righe del dataset:")
print(df_student.head())

x_raw = df_student['YearsExperience'].values
y_raw = df_student['Salary'].values

plt.figure(figsize=(12, 8))
plt.scatter(x_raw, y_raw, color='red', label='Dati Reali (Salary)', s=50,
            zorder=5)

gradi_da_testare = [1, 3, 12]

x_min, x_max = x_raw.min(), x_raw.max()
xx = np.linspace(x_min - 0.5, x_max + 0.5, 1000)

colors = ['green', 'red', 'blue']

for i, d in enumerate(gradi_da_testare):
    X = vandermonde(x_raw, d)
    alpha = SVD(X, y_raw)
    yy_svd = f(xx, alpha)
    plt.plot(xx, yy_svd, linewidth=2, color=colors[i], label=f'd={d}')

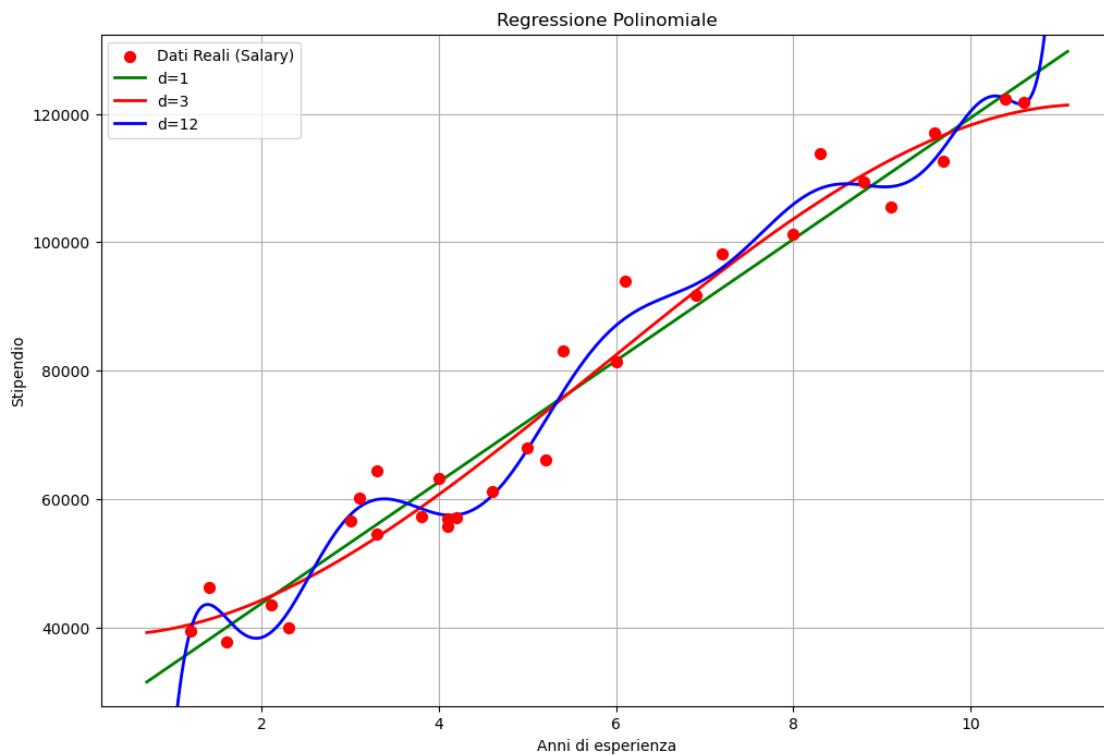
plt.title("Regressione Polinomiale")
plt.xlabel("Anni di esperienza")
plt.ylabel("Stipendio")
plt.legend()
```

```
plt.grid(True)
plt.ylim(y_raw.min() - 10000, y_raw.max() + 10000)
```

Prime 5 righe del dataset:

	Unnamed: 0	YearsExperience	Salary
0	0	1.2	39344.0
1	1	1.4	46206.0
2	2	1.6	37732.0
3	3	2.1	43526.0
4	4	2.3	39892.0

[14]: (27732.0, 132392.0)



2 Regressione Multipla

```
[6]: nome_file = 'Student_Performance.csv'
df_student = pd.read_csv(nome_file)

x_raw = df_student.drop("Performance Index", axis=1)
```

```

x_raw["Extracurricular Activities"] = x_raw["Extracurricular Activities"].
    ↪map({'Yes': 1, 'No':0})
y_raw = df_student["Performance Index"]

X_train, X_test, y_train, y_test = train_test_split(x_raw, y_raw, test_size=0.
    ↪2, random_state=42)

model = LinearRegression()

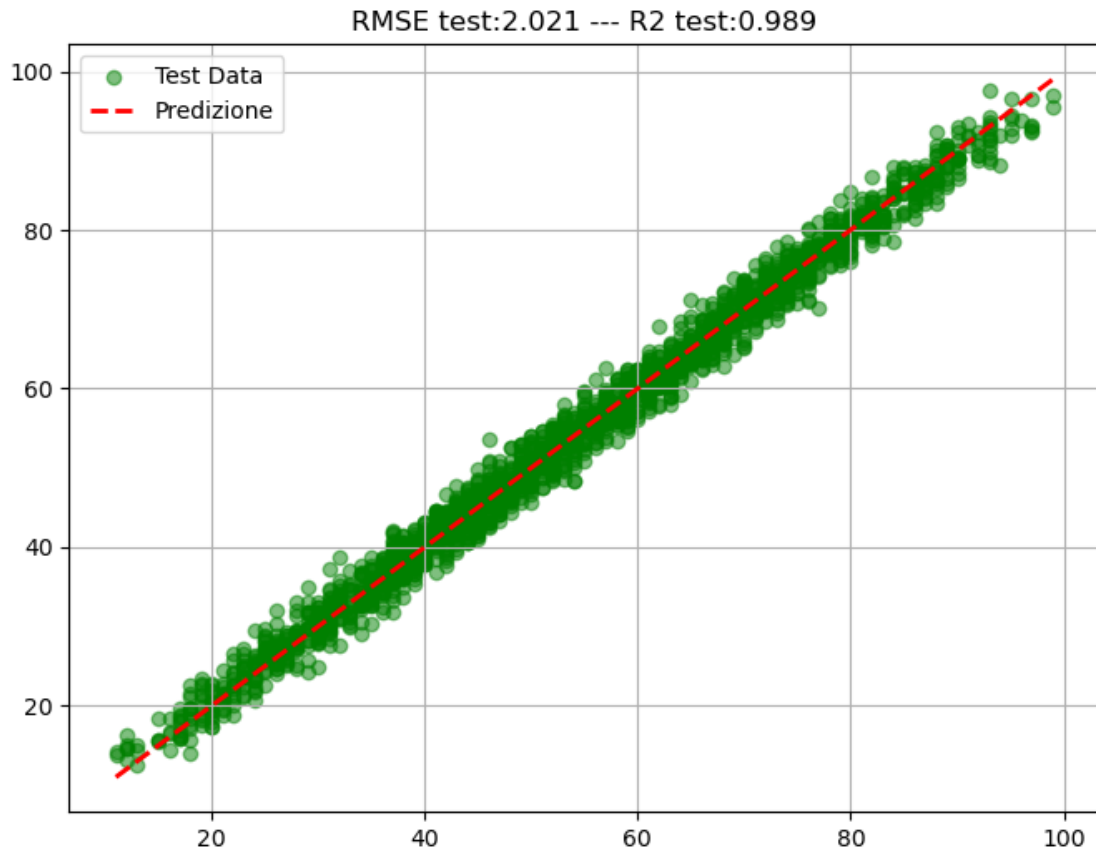
model.fit(X_train, y_train)

y_pred_test = model.predict(X_test)
y_pred_train = model.predict(X_train)

rmse_test = np.sqrt(mean_squared_error(y_test, y_pred_test))
r2_test = r2_score(y_test, y_pred_test)

plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred_test, color='green', alpha=0.5, label='Test Data')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--',
    ↪lw=2, label='Predizione')
plt.title(f"RMSE test:{rmse_test:.3f} --- R2 test:{r2_test:.3f}")
plt.legend()
plt.grid(True)
plt.show()

```



```
[7]: feature_names = x_raw.columns

coefficients = model.coef_

plt.figure(figsize=(10, 6))

plt.barh(feature_names, coefficients)
plt.axvline(0, color='black')

plt.title("Quanto influisce ogni fattore sul voto finale?")
plt.xlabel("Alpha")
plt.grid(axis='x', linestyle='--', alpha=0.7)

for index, value in enumerate(coefficients):
    plt.text(value, index, f'{value:.2f}', va='center')
```

